open-cam: an open-source hyperspectral, ray-traced camera simulator built on pbrt-v4 for sensor, lens and CFA studies

Brian Deegan; University of Galway; Galway, Ireland

Abstract

We present open-cam, an open-source (MIT) camera simulator that couples the physically based renderer pbrt-v4 with a radiometrically explicit sensor model. Unlike most driving and robotics simulators, open-cam never forms an RGB image inside the renderer: pbrt-v4 writes per-wavelength radiance buckets, which are converted to sensor-plane irradiance, photon counts and photoelectrons with measured quantum efficiency and infrared-cut-filter curves, then passed through an EMVA1288-style noise, ADC and colour filter array model. Lenses can be traced through real prescriptions. We describe the architecture, compare it with open-source and commercial camera simulators, and report reproducible validation: an end-to-end ColorChecker render, a photon-transfer check that recovers the configured system gain to within 0.3%, a traced-lens slanted-edge MTF, and a study of non-Bayer filter arrays derived from a single spectral render. We are explicit about current limitations, notably that only Bayer mosaics are sampled and demosaiced end-to-end.

Introduction

Camera simulation is used to prototype sensors, to evaluate image signal processors (ISPs) and to generate training and test data for perception. Two families of tools exist. Image-systems simulators such as ISETCam [1, 2, 3] model the physics of light, optics and sensors in spectral units, while driving and robotics simulators (CARLA [4], AirSim [5], NVIDIA Isaac Sim [6] and commercial platforms) emphasise real-time scene dynamics and typically render RGB images to which camera effects are added afterwards.

Many questions that matter in automotive and mobile imaging cannot be answered from an RGB render: how a red–clear–clear–blue (RCCB) or red–yellow–yellow–cyan (RYYCy) mosaic changes low-light signal-to-noise ratio (SNR), how the infrared-cut filter (IRCF) interacts with an LED or tungsten illuminant, or how a lens with longitudinal chromatic aberration blurs each colour channel. All require the spectral radiance at the sensor. open-cam [7] is a Python toolkit built to answer such questions with free tools. Its contributions are: (i) a spectral pipeline from pbrt-v4 [8, 9] spectral film to photoelectrons, with every radiometric factor explicit; (ii) an EMVA1288-style [10] sensor model with analytic validators and real-renderer end-to-end tests; (iii) a library of camera recipes, measured spectral data and traced lens prescriptions; and (iv) an honest account of what is and is not yet implemented, in particular for non-Bayer colour filter arrays (CFAs).

Related work

Image-systems simulation. ISETCam (MATLAB, MIT licence) keeps scene spectral radiance, optical-image irradiance and sensor photoelectrons and voltages in physical units, and includes models of CFAs, optics and displays [3, 2]. ISET3D (MIT) adds 3D rendering with pbrt-v4 [11]; it has been validated against measured cameras [12, 13] and extended with ray-transfer functions for lenses whose design is hidden [14]. ISETAuto (Apache-2.0) applies the stack to driving scenes and network evaluation [15, 16, 17]. open-cam is closest to this family; it differs in being pure Python, in its configuration-driven catalogue of 73 camera recipes, and in its EMVA1288-oriented validators and continuous-integration tests that run the real renderer. It does not yet offer the breadth of ISETCam's optics, display and human-vision models.

Renderers. pbrt-v4 (Apache-2.0) renders with spectral sampling and provides a `spectral` film that stores radiance in wavelength buckets, as well as a `realistic` camera that traces a lens prescription [8, 18, 19]. Mitsuba 3 (BSD-style licence) offers spectral and polarised variants [20, 21]. Both are renderers, not camera models: they stop at radiance or at a colour image.

Driving and robotics simulators. CARLA (MIT) renders RGB cameras with Unreal Engine post-processing; its documented camera attributes include auto exposure, bloom, grain jitter and lens-distortion parameters [22]. AirSim (MIT) exposes noise and distortion settings for its RGB cameras [23]; Gazebo models camera noise as independent Gaussian noise per pixel [24]. NVIDIA Isaac Sim's RTX camera documents OpenCV pinhole and fisheye distortion models, additive noise, encoding of the RGB image into a single-channel Bayer mosaic and an on-chip ISP stage [6]; NVIDIA's DRIVE simulation stack advertises high-fidelity sensor simulation [25]. None of these documentations describe spectral rendering into a QE-weighted sensor.

Commercial sensor and optics tools. Ansys Speos performs spectral, wavelength-dependent ray tracing with camera and detector response simulation [26, 27]; Ansys AVxcelerate Sensors provides a real-time, physics-based camera model parameterised from a camera datasheet and a multispectral material database [28]. Lens-design packages (Ansys Zemax OpticStudio [29], Synopsys LightTools [30]) provide sequential and non-sequential ray tracing and stray-light analysis. Scenario-level platforms, including Siemens Simcenter Prescan [31], IPG CarMaker [32], Hexagon VTD [33], rFpro [34], Cognata [35] and dSPACE AURELION, describe "physics-based" or ray-traced sensor simulation; rFpro, for example, documents a multi-exposure HDR camera API with motion blur [34]. Foretellix [36] targets scenario generation and coverage rather than sensor physics. For these commercial tools the spectral resolution, noise model and CFA options are not specified in the public material we could access, so Table 1 marks them as not publicly documented rather than guessing. Pricing is quote-based throughout.

Measurement. EMVA1288 [10, 37], photon transfer [38] and ISO 12233 slanted-edge analysis [39, 40] define how real cameras are characterised; the EMVA1288 reference implementation (LGPL-3.0) [41] and commercial suites such as Imatest [42] implement them for measured data. open-cam uses the same definitions so that simulated and measured cameras can be compared on equal terms.

Camera-simulation capabilities as stated in each tool's public documentation or papers (accessed October 2026). "n.d." = not documented in the public sources we reviewed; it does not mean the capability is absent.

Tool	Spectral rendering	Ray-traced lens	Sensor noise / EMVA1288	CFA flexibility	Licence / cost
open-cam	Yes: pbrt-v4 spectral film, user-set buckets	Yes: pbrt-v4 realistic camera, 4 prescriptions	Shot, read, PRNU, DSNU, dark current, ADC, FPN, defects; EMVA validators	Arbitrary QE curves; Bayer (4 phases) end-to-end only	MIT; free
ISETCam / ISET3D / ISETAuto	Yes (spectral radiance)	Yes (pbrt-v4; ray-transfer functions)	Physical photoelectron/voltage model	CFA models included [3]	MIT / Apache-2.0; MATLAB required
pbrt-v4, Mitsuba 3	Yes	pbrt-v4: realistic camera	None (renderer)	None	Apache-2.0 / BSD; free
CARLA, AirSim, Gazebo	No (RGB)	No; distortion parameters	Grain / Gaussian / noise-line effects	None (RGB output)	MIT / Apache-2.0; free
Isaac Sim (RTX)	n.d.	No; OpenCV distortion models	Additive noise	Bayer encoding + ISP	NVIDIA licence
Ansys Speos	Yes	Yes	Detector response, raw signal	n.d.	Commercial
Ansys AVxcelerate	Multispectral materials	n.d.	Datasheet-parameterised	n.d.	Commercial
Zemax OpticStudio, LightTools	Wavelength-dependent tracing	Yes (lens design)	n.d.	n.d.	Commercial
Prescan, CarMaker, VTD, rFpro, Cognata, AURELION, DRIVE Sim	n.d.	n.d. (rFpro: ray tracing)	n.d.	n.d.	Commercial
Imatest, emva1288	– (measurement)	–	EMVA1288 analysis of measured data	–	Commercial / LGPL-3.0

Architecture

open-cam is a set of command-line tools orchestrated by `tools/run_pipeline.py`. A single YAML file (`config/pipeline.yaml`) selects a camera recipe and drives four stages: scene generation, pbrt-v4 rendering, spectral-to-electron conversion and the sensor/noise/CFA stage, followed by validators. A recipe in `config/camera_recipes/` (73 are provided: phone, compact, interchangeable-lens and research cameras) references a sensor model (QE curves, pixel pitch, fill factor, exposure, EMVA parameters, CFA) and a lens model. Sensor models are written as deltas from `config/sensor_models/default.yaml`, so a new sensor needs only its overrides.

Scenes. `build_colorchecker_scene.py` builds a 24-patch ColorChecker from measured X-Rite reflectances lit by a tabulated illuminant (D50–D75, A, C, fluorescent and LED spectra are included). `build_image_quality_targets.py` generates slanted-edge, Siemens-star and ISO-style noise targets, and `build_munsell_scenes.py` builds charts from 1269 measured Munsell chips (380–800 nm at 1 nm).

Spectral rendering. Scenes are rendered with pbrt-v4's `spectral` film, which writes an OpenEXR file with one radiance channel per wavelength bucket (`S0.`$\langle\lambda\rangle$`nm`, in $\mathrm{W\,m^{-2}\,sr^{-1}\,nm^{-1}}$) [18]. The default pipeline uses 64 buckets between 360 and 830 nm. Cameras can be pinhole, thin-lens or pbrt-v4's `realistic` camera, which traces rays through a lens prescription [19]. The repository ships four prescriptions (a 22 mm wide-angle, a 50 mm double-Gauss, a 250 mm telephoto and a 10 mm fisheye) and 14 lens models that bind them, or thin-lens parameters, to camera classes.

Radiance to electrons. Radiance is converted to sensor-plane irradiance by `pbrt_spectral_exr_to_electrons.py` using the camera equation

$$E_\lambda(x,y) = L_\lambda(x,y)\,\frac{\pi\,\tau(\lambda)}{4N^2(1+m)^2}\,c_{\mathrm{cal}}, \tag{1}$$

where N is the f-number (taken from the traced prescription when the realistic camera is used), m the magnification, τ the optics transmittance and c_{cal} an optional photometric calibration to a target scene illuminance. The mean electrons in channel c are then

$$\mu_{e,c} = \sum_\lambda E_\lambda \frac{\lambda}{hc}\,\eta_c(\lambda)\,T_{\mathrm{IR}}(\lambda)\,\Delta\lambda\;A_{\mathrm{px}}\,t\,F\,V_c(x,y), \tag{2}$$

with QE η_c, IRCF transmittance T_{IR}, bucket width $\Delta\lambda$, pixel area A_{px}, exposure time t, fill factor F and an optional vignetting map V_c. The photon conversion, QE, bin widths and geometry are folded into a $K \times C$ weight matrix (`sensor_radiometry.spectral_electron_weights`), so integration is a single matrix product per pixel. The same helper is shared with `spectral_sensor_forward.py`, an analytic forward model used for fast tests. QE curves for many cameras are taken from a measured spectral-sensitivity database; generic red, green, blue, cyan, yellow, magenta and monochrome curves and an IRCF are also provided.

Sensor model. `apply_emva_noise.py` implements the EMVA1288 linear camera model [10]: Poisson shot noise on signal plus dark current (with a temperature model), PRNU as a multiplicative gain map, DSNU as an additive offset map, Gaussian read noise, optional kTC noise, row/column fixed-pattern and flicker noise, hot/stuck pixels, blooming, inter-pixel crosstalk, full-well clipping and an ADC with gain K ($\mathrm{e^-}$/DN), black level, bit depth and optional

INL/DNL. Spatial maps use a fixed seed (one "camera unit") while temporal noise uses a per-frame seed. A Bayer mosaic is sampled with any of the four RGGB/BGGR/GRBG/GBRG phases and demosaiced bilinearly or with the Malvar–He–Cutler algorithm [43]. `apply_spectral_psf.py` adds post-render optics where a traced lens is not used: Gaussian, wavelength-dependent Gaussian and Airy-disk PSFs per spectral bucket, lateral chromatic aberration and stray light.

Validation and tests. `validate_emva_model.py` compares the analytic EMVA mean and variance with Monte Carlo at photon-transfer levels and checks the recipe against its datasheet block; `batch_validate_emva.py` runs this across all recipes. `validate_demosaic_linear.py` checks demosaic fidelity in linear DN. Twenty test modules include two that call the real pbrt-v4 binary: `test_pbrt_e2e.py` and `test_pbrt_full_pipeline.py`. The latter renders a 96×64, 16-bucket ColorChecker and, for every pixel, forms the residual $z = (\mathrm{DN} - (b + \mu_e/K))/\sigma_{\mathrm{pred}}$ against the analytic EMVA prediction, requiring zero mean and unit variance; it also checks ADC limits, seed determinism and demosaic error.

Teaching. A Dear PyGui [44] desktop application (`gui/`) exposes the same engines through interactive scenarios on optics, PSF and aberrations, MTF, sensor noise, exposure and defects, colour/ISP and image generation, with accompanying tutorials.

Results

All results below were produced on a CPU build of pbrt-v4 with the scripts in `paper/ei2027/scripts/`; the exact commands are listed in the accompanying README. Renders are 960×640.

End-to-end ColorChecker. We rendered the ColorChecker with the geometry and lens of `config/pipeline.yaml` (realistic 22 mm wide-angle prescription, 8.756 mm aperture, focus 4 m, 1024 samples per pixel, 64 buckets over 360–830 nm) under CIE D65 and CIE A, and ran it through the `iphone_8` recipe (1.4 µm pixels, GBRG, Malvar demosaic, $K = 9.906\,\mathrm{e}^-/\mathrm{DN}$, $9500\,\mathrm{e}^-$ full well, 10-bit) at 500 lux and 0.12 s, the pipeline defaults. With the realistic camera the effective f-number from the prescription is f/2.51. The white patch yields 364/821/439 e^- (R/G/B) under D65. Fig. 1 shows the demosaiced output with grey-world preview white balance; the gains change from 2.19/1.00/1.83 (D65) to 1.27/1.00/3.21 (A), a direct consequence of integrating each illuminant's spectrum against the camera QE rather than of an RGB tint. Against a noise-free reference, `validate_demosaic_linear.py` reports a Malvar demosaic error of 0.114 DN MAE and 0.389 DN RMSE.

Photon transfer. Using the iphone_8 EMVA parameters ($\sigma_d = 1.6\,\mathrm{e}^-$, PRNU 1.8%, DSNU $0.16\,\mathrm{e}^-$), we simulated 256×256 uniform frame pairs at 29 exposure levels from dark to beyond saturation with `emva_theory.simulate_uniform_stack`, and applied the EMVA1288 photon-transfer method (temporal variance from frame-pair differences; zero-intercept fit of $\sigma_y^2 - \sigma_{y,\mathrm{dark}}^2$ against $\mu_y - \mu_{y,\mathrm{dark}}$ up to 70% of full well). The fit recovers $K = 9.883\,\mathrm{e}^-/\mathrm{DN}$ against the configured 9.906 (0.23% error) and $\sigma_d = 1.588\,\mathrm{e}^-$ (configured 1.6). The EMVA PRNU estimate from 16-frame dark and 50%-saturation stacks is 1.79% (configured 1.8%). Across the linear range the simulated variance deviates from the analytic model by at most 1.3% (median 0.46%), consistent with the sampling error of a 65,536-pixel estimate (Fig. 2). `validate_emva_model.py` passes all photon-transfer, dark-frame and datasheet checks for this recipe with 20,000 Monte Carlo trials.

Traced-lens MTF. We rendered a 5° slanted edge at 3.2 m through the 50 mm double-Gauss prescription (32 buckets, 400–700 nm) at f/2 in focus, at f/2 focused at 2.6 m, and at f/5.6 focused at 2.6 m, and analysed it with `sfr_analysis.slanted_edge_sfr`. With the generic green QE the MTF50 is 0.209, 0.270 and 0.294 cy/px respectively, and the in-focus MTF at Nyquist is 0.127 (Fig. 3). Two observations are worth reporting as measured rather than explained away. First, the nominal `focusdistance` of 3.2 m is not the sharpest setting: focusing at 2.6 m gives a higher MTF50, so a through-focus sweep, rather than the nominal value, should be used to find best focus with this prescription. Second, stopping down to f/5.6 raises MTF50 further, as expected from the smaller blur circle. Because the film is spectral, the same render also yields an MTF per wavelength bucket (Fig. 3, right). Here we report the data only with a caveat: the per-bucket MTF50 shows a periodic ripple of roughly ±5% with a period of about five buckets in all three renders, which is not plausible as a lens property and which we have not yet traced (pbrt-v4's stochastic wavelength sampling is a candidate). The bucket-to-bucket structure should therefore not be interpreted as chromatic aberration; resolving it, e.g. with monochromatic renders, is required before per-wavelength MTF can be used quantitatively. Edge positions were located with a 50%-crossing fit after per-row flat-fielding, because the derivative-centroid estimator in `sfr_analysis.py` was biased by render noise on this edge (it reported a −2.3° edge for a measured −5.0°); the ESF binning, LSF and MTF use the repository functions. Note that these values are in film pixels of the renderer and include the pbrt-v4 Gaussian reconstruction filter.

CFA support: what exists. We inspected the CFA path to state its scope precisely. `apply_emva_noise.py` samples and demosaics only 2×2 Bayer phases and rejects other pattern names. The recipes named `default_rccb`, `default_ryycy`, `default_rccg`, `default_rggcy` and `default_cmy` are *RGB proxies*: the three QE slots of the Bayer path are filled with other curves (for RCCB, the "clear" channel is the generic cyan curve in the green slot; for RYYCy, yellow and cyan replace green and blue), and the result is still sampled on a Bayer lattice. RGBW, true clear pixels, quad-Bayer [45] and arbitrary $n \times n$ tiles are therefore not yet supported end-to-end, and there is no non-Bayer demosaicing.

CFA support: what the spectral design enables. Because the renderer output is spectral, any channel set can nonetheless be computed from the same EXR with the pipeline's own integration weights, and sampled on any tile. `fig_cfa.py` does this outside the pipeline (no pipeline code is modified): it integrates the D65 and A ColorChecker renders against six QE×IRCF curves (Fig. 4), with "W" the unfiltered monochrome silicon QE, and anchors the absolute scale to the pipeline's green-channel electrons for the default recipe (the ratio is constant across all 24 patches to a coefficient of variation of 4×10^{-8}, confirming that the script and the pipeline implement the same radiometry). Fig. 5 shows raw mosaics for six layouts. Table 2 reports, for the neutral 5 patch (about 19% reflectance), the luminance-channel electrons at 500 lux and the analytic EMVA temporal SNR, $\mu_e/\sqrt{\mu_e + \sigma_d^2}$ with $\sigma_d = 2.6\,\mathrm{e}^-$, scaled to 10 lux. A clear pixel collects 2.62× the electrons of the green pixel and gains 6.0 dB of SNR in this low-light case; yellow and the cyan proxy gain 2.8 and 3.2 dB. These ratios depend on the illuminant: under CIE A the yellow/green ratio rises from 1.55 to 2.12 and the cyan/green ratio falls from 1.65 to 1.11. The spectral render also quantifies the error of the current RGB proxy: normalised on the white patch, the cyan-proxy/clear ratio ranges from 0.43 (patch 15, red) to 1.34 (patch

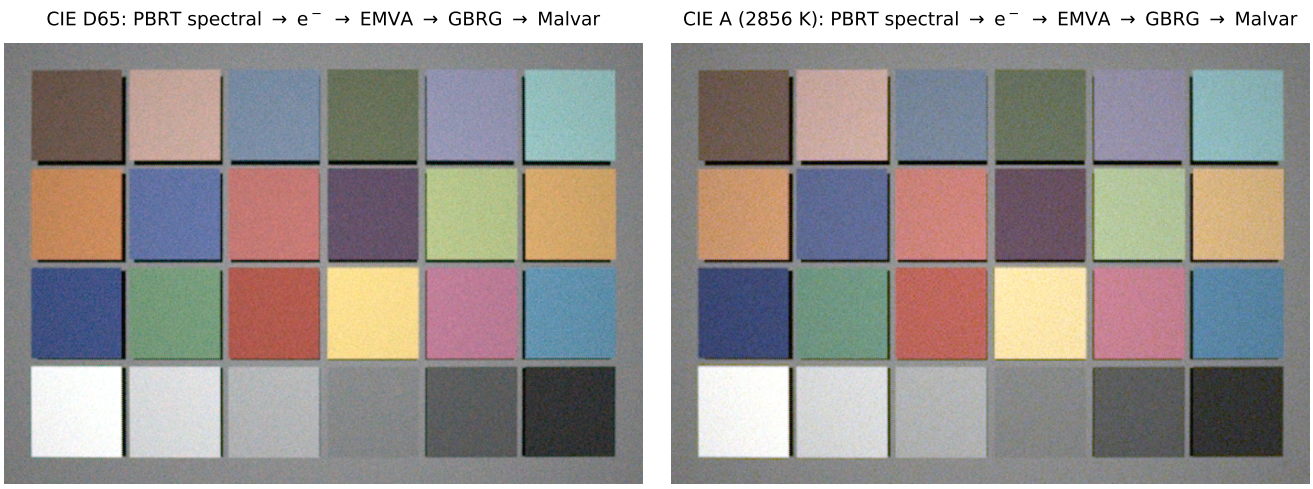


Figure 1. *ColorChecker rendered spectrally by pbrt-v4 (64 buckets) and processed by open-cam with the iphone_8 recipe: electrons → EMVA noise → GBRG mosaic → Malvar demosaic, grey-world preview white balance, no colour correction. Left: D65; right: CIE A.*

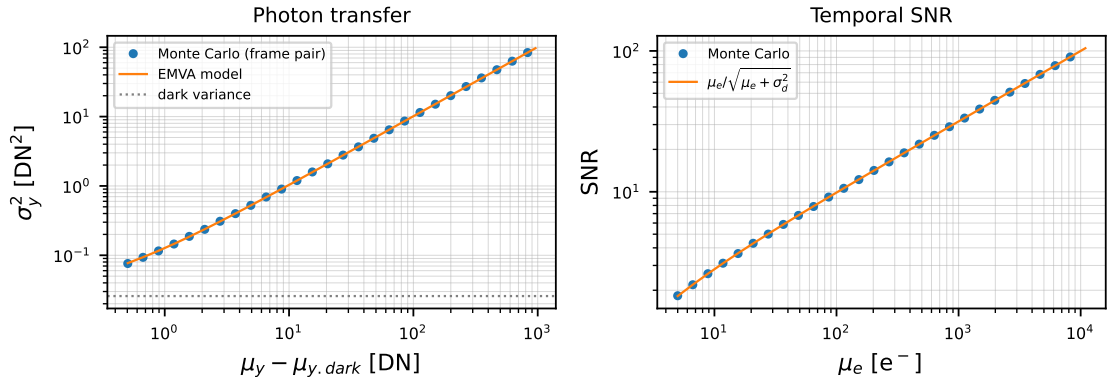


Figure 2. *EMVA1288 photon-transfer validation for the iphone_8 recipe. Left: temporal variance versus dark-subtracted mean signal for Monte Carlo frame pairs (markers) and the analytic model (line); the gain fit uses levels up to 70% of full well. Right: temporal SNR versus mean electrons.*

Luminance-channel signal on the neutral 5 patch from one D65 render (default recipe, 0.12 s) and analytic temporal SNR at 10 lux.

CFA (channel)	μ_e @500 lx	SNR @10 lx	Δ	ch/G (A)
Bayer / quad (G)	282	4.1 dB	–	1.00
RGBW, RCCB (W)	741	10.1 dB	+6.0 dB	2.67
RCCB proxy (Cy)	465	7.3 dB	+3.2 dB	1.11
RYYCy (Ye)	438	6.9 dB	+2.8 dB	2.12

13, blue), so a cyan curve is not a scalar stand-in for a clear pixel, and the error depends on scene colour. Quad-Bayer has identical per-pixel statistics to Bayer here because binning and remosaicing are not modelled. Per-pixel analyses of this kind are impossible from an RGB render, which is why we regard the spectral representation, rather than the current CFA code, as open-cam's main asset for CFA research [46].

Discussion and limitations

The results support the central design choice: keeping light spectral until the photon-counting step makes illuminant, IRCF and filter questions first-class, and the EMVA validators make the noise model auditable against datasheets. The limitations are equally concrete. Non-Bayer CFAs are not sampled or demosaiced in the pipeline; the natural extension is to generalise the Bayer lookup tables in `apply_emva_noise.py` to an $n \times n$ tile of named QE curves with per-tile demosaicing, which the spectral integration already supports. Many recipes carry EMVA parameters inferred from public tables (each recipe records a calibration tier) rather than measured reports. Rendering is offline: the 1024-sample, 64-bucket ColorChecker took roughly 4.5 min on an 8-core CPU, so open-cam complements rather than replaces real-time simulators. Scenes are currently charts and targets rather than driving environments, and there is no motion blur, rolling shutter or HDR multi-exposure model yet.

Conclusion

open-cam provides a free, scriptable path from pbrt-v4 spectral radiance to EMVA-consistent raw data through traced lenses

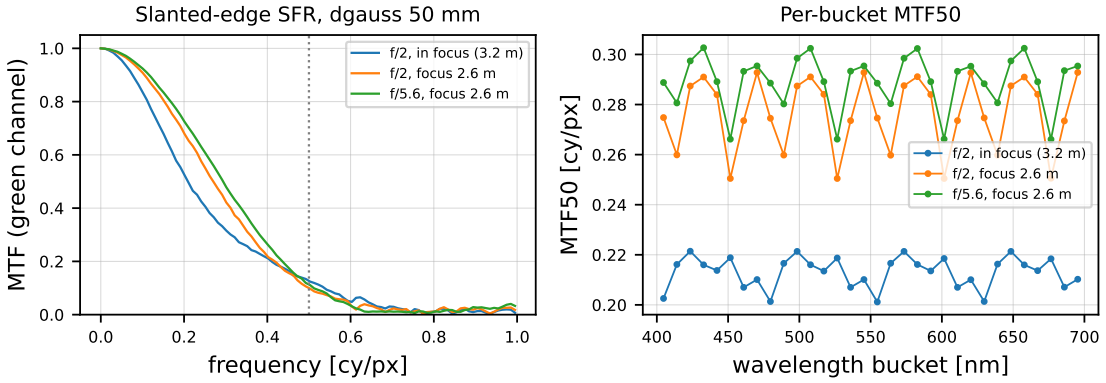


Figure 3. *Slanted-edge MTF through the traced 50 mm double-Gauss lens. Left: green-channel MTF for three aperture/focus settings (dotted line: Nyquist). Right: MTF50 computed separately in each spectral bucket of the same renders; the periodic ripple is unexplained (see text).*

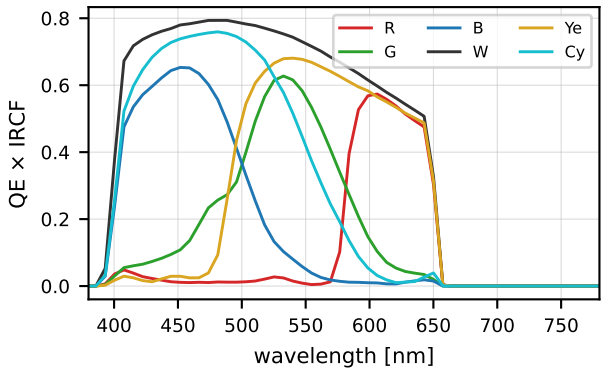


Figure 4. *Generic QE×IRCF curves shipped in* `spectra/QE/interpolated/` *and used for the CFA study (W: unfiltered monochrome QE).*

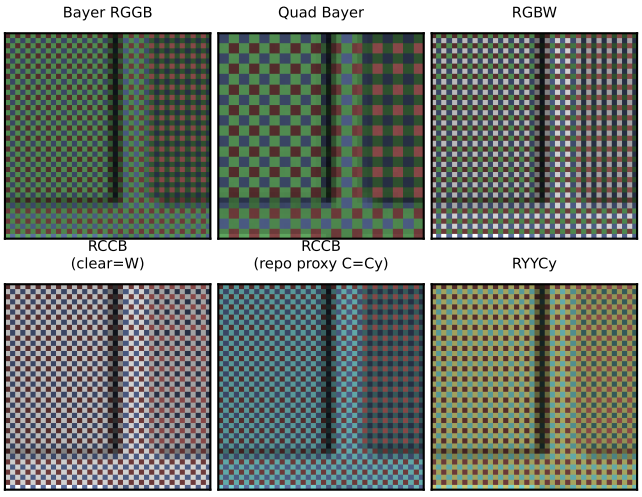


Figure 5. *Raw mosaics computed from one spectral render (D65) at the corner of patches 14, 15, 20 and 21, each pixel tinted by its filter. Only Bayer is sampled by the pipeline itself; the other layouts are produced by* `fig_cfa.py` *from the same spectral buckets.*

and measured QE curves. It recovers configured sensor parameters through the EMVA1288 photon-transfer method, reproduces illuminant-dependent white balance from first principles, and, while its pipeline CFA support is currently Bayer-only, its spectral representation already allows quantitative studies of RGBW, RCCB and RYYCy designs. Code, configurations and all scripts used for this paper are available at `https://github.com/briandeegan82/open-cam`.

References

[1] J. E. Farrell, F. Xiao, P. B. Catrysse, and B. A. Wandell, "A simulation tool for evaluating digital camera image quality," in *Proc. SPIE 5294, Image Quality and System Performance*, pp. 124–131, 2003.

[2] J. E. Farrell, P. B. Catrysse, and B. A. Wandell, "Digital camera simulation," *Applied Optics*, vol. 51, no. 4, pp. A80–A90, 2012.

[3] "ISETCam (MIT licence)." `https://github.com/ISET/isetcam`. Accessed 2026-10-08.

[4] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. 1st Annual Conference on Robot Learning (CoRL), PMLR 78*, pp. 1–16, 2017.

[5] S. Shah, D. Dey, C. Lovett, and A. Kapoor, "AirSim: High-fidelity visual and physical simulation for autonomous vehicles," in *Field and Service Robotics*, Springer Proceedings in Advanced Robotics, 2017.

[6] NVIDIA, "Isaac Sim documentation: Camera sensors." `https://docs.isaacsim.omniverse.nvidia.com/latest/sensors/isaacsim_sensors_camera.html`. Accessed 2026-10-08.

[7] B. Deegan, "open-cam: spectral pbrt-v4 camera simulation." `https://github.com/briandeegan82/open-cam`. Accessed 2026-10-08.

[8] M. Pharr, W. Jakob, and G. Humphreys, *Physically Based Rendering: From Theory to Implementation*. Cambridge, MA: MIT Press, 4th ed., 2023.

[9] M. Pharr *et al.*, "pbrt-v4 source code (Apache-2.0)." `https://github.com/mmp/pbrt-v4`. Accessed 2026-10-08.

[10] European Machine Vision Association, "EMVA Standard 1288: Standard for Characterization of Image Sensors and Cameras, Release 4.0." `https://www.emva.org/standards-technology/emva-1288/`, 2021. Accessed 2026-10-08.

[11] "ISET3D: pbrt-v4 based 3D rendering for ISETCam (MIT licence)." `https://github.com/ISET/iset3d`. Accessed 2026-10-08.

[12] Z. Lyu, K. Kripakaran, M. Furth, E. Tang, B. Wandell, and J. Farrell,

"Validation of image systems simulation technology using a Cornell box," in *Proc. IS&T Electronic Imaging: Imaging Sensors and Systems*, 2021.

[13] Z. Lyu, T. Goossens, B. A. Wandell, and J. Farrell, "Validation of physics-based image systems simulation with 3-d scenes," *IEEE Sensors Journal*, vol. 22, no. 20, pp. 19400–19410, 2022.

[14] T. Goossens, Z. Lyu, J. Ko, G. C. Wan, J. Farrell, and B. Wandell, "Ray-transfer functions for camera simulation of 3D scenes with hidden lens design," *Optics Express*, vol. 30, no. 13, p. 24031, 2022.

[15] "ISETAuto (Apache-2.0 licence)." `https://github.com/ISET/isetauto`. Accessed 2026-10-08.

[16] H. Blasinski, J. Farrell, T. Lian, Z. Liu, and B. Wandell, "Optimizing image acquisition systems for autonomous driving," in *Proc. IS&T Electronic Imaging: Photography, Mobile, and Immersive Imaging*, 2018.

[17] Z. Liu, T. Lian, J. Farrell, and B. A. Wandell, "Soft prototyping camera designs for car detection based on a convolutional neural network," in *Proc. IEEE/CVF International Conference on Computer Vision Workshops (ICCVW)*, 2019.

[18] "pbrt-v4 User's Guide and File Format." `https://pbrt.org/fileformat-v4`. Accessed 2026-10-08.

[19] C. Kolb, D. Mitchell, and P. Hanrahan, "A realistic camera model for computer graphics," in *Proc. SIGGRAPH '95*, pp. 317–324, 1995.

[20] M. Nimier-David, D. Vicini, T. Zeltner, and W. Jakob, "Mitsuba 2: A retargetable forward and inverse renderer," *ACM Transactions on Graphics*, 2019.

[21] "Mitsuba 3 documentation: Choosing variants." `https://mitsuba.readthedocs.io/en/stable/src/key_topics/variants.html`. Accessed 2026-10-08.

[22] "CARLA documentation: Sensors reference." `https://carla.readthedocs.io/en/latest/ref_sensors/`. Accessed 2026-10-08.

[23] "AirSim documentation: Settings." `https://microsoft.github.io/AirSim/settings/`. Accessed 2026-10-08.

[24] "Gazebo Classic tutorial: Sensor noise model." `https://classic.gazebosim.org/tutorials?tut=sensor_noise`. Accessed 2026-10-08.

[25] NVIDIA, "Simulation for Self-Driving Vehicles." `https://www.nvidia.com/en-us/self-driving-cars/simulation/`. Accessed 2026-10-08.

[26] Ansys, "Ansys Speos." `https://www.ansys.com/products/optics/ansys-speos`. Accessed 2026-10-08.

[27] M. Falahati, V. Loya, S. Kalkhoran, and V. Singh, "Modeling and evaluation of ADAS camera jitter due to road-induced random vibration using Ansys Speos," in *Proc. SPIE Optical Architectures for Displays and Sensing in Augmented, Virtual, and Mixed Reality*, 2026.

[28] Ansys, "Ansys AVxcelerate Sensors." `https://www.ansys.com/products/av-simulation/ansys-avxcelerate-sensors`. Accessed 2026-10-08.

[29] Ansys, "Ansys Zemax OpticStudio." `https://www.ansys.com/products/optics/ansys-zemax-opticstudio`. Accessed 2026-10-08.

[30] Synopsys, "LightTools illumination design software." `https://www.synopsys.com/optical-solutions/lighttools.html`. Accessed 2026-10-08.

[31] Siemens, "Simcenter Prescan." `https://plm.sw.siemens.com/en-US/simcenter/autonomous-vehicle-solutions/prescan/`. Accessed 2026-10-08.

[32] IPG Automotive, "CarMaker." `https://www.ipg-automotive.com/en/products-solutions/software/carmaker/`. Accessed 2026-10-08.

[33] S. Son, W. Lee, H. Jung, J. Lee, C. Kim, H. Lee, H. Park, H. Lee, J. Jang, S. Cho, and H.-C. Ryu, "Evaluation of camera recognition performance under blockage using virtual test drive toolchain," *Sensors*, vol. 23, no. 19, p. 8027, 2023.

[34] rFpro, "Ray tracing." `https://rfpro.com/simulation-software/ray-tracing/`. Accessed 2026-10-08.

[35] Cognata, "Cognata." `https://www.cognata.com/`. Accessed 2026-10-08.

[36] Foretellix, "Foretellix." `https://www.foretellix.com/`. Accessed 2026-10-08.

[37] M. E. Borek, "Implementation of EMVA 1288 standard release 4.0 for characterization of image sensors," in *Proc. IS&T Electronic Imaging: Imaging Sensors and Systems*, 2023.

[38] J. R. Janesick, *Photon Transfer*. SPIE Press, 2007.

[39] ISO, "ISO 12233:2017 Photography – Electronic still picture imaging – Resolution and spatial frequency responses." International Organization for Standardization, 2017.

[40] D. Williams and P. D. Burns, "Low-frequency MTF estimation for digital imaging devices using slanted-edge analysis," in *Proc. SPIE 5294, Image Quality and System Performance*, pp. 93–101, 2003.

[41] "EMVA1288 reference implementation (LGPL-3.0)." `https://github.com/EMVA1288/emva1288`. Accessed 2026-10-08.

[42] Imatest, "Imatest image quality testing software." `https://www.imatest.com/`. Accessed 2026-10-08.

[43] H. S. Malvar, L. He, and R. Cutler, "High-quality linear interpolation for demosaicing of Bayer-patterned color images," in *Proc. IEEE ICASSP*, 2004.

[44] "Dear PyGui." `https://github.com/hoffstadt/DearPyGui`. Accessed 2026-10-08.

[45] T. Okawa, S. Ooki, H. Yamajo, M. Kawada, M. Tachi, K. Goi, T. Yamasaki, H. Iwashita, M. Nakamizo, T. Ogasahara, Y. Kitano, and K. Tatani, "A 1/2inch 48M all PDAF CMOS image sensor using 0.8μm Quad Bayer coding 2×2OCL with 1.0lux minimum AF illuminance level," in *Proc. IEEE International Electron Devices Meeting (IEDM)*, 2019.

[46] K. Hirakawa and P. J. Wolfe, "Spatio-spectral color filter array design for optimal image recovery," *IEEE Transactions on Image Processing*, 2008.

Author Biography

Brian Deegan received bachelor's (computer engineering, 2004) and MSc (biomedical engineering, 2005) degrees from the University of Limerick, and a PhD in biomedical engineering from NUI Galway (2011). From 2011 to 2022 he was a vision research engineer at Valeo Vision Systems, working on HDR imaging, LED flicker and image quality for machine vision, and from 2014 a Valeo Expert on Image Quality. In 2022 he joined the University of Galway as a lecturer in Electrical & Electronic Engineering.